

EXPERIMENT NO: 2

Name of the student: Shraddha Gajanan Desai

Roll No: 18

Division: A

Batch: A

Aim of the Experiment: Write a program to implement Diffie-Hellman Algorithm.

Learning Objectives: To understand the principles of symmetric key cryptography.

To understand the Diffie-Hellman Key exchange algorithm.

To understand the possible attacks on Diffie-Hellman.

Lab Outcomes: Apply the cryptosystem to ensure secure key exchange between sender and receiver.

Tools & Software Required: Python

Theory:

(Explain following terms with one example)

- **Diffie-Hellman key Exchange Algorithm:**

Diffie–Hellman Key Exchange Algorithm

Diffie–Hellman is a **cryptographic key exchange method** that allows two users to **securely generate a shared secret key** over an insecure channel.

Steps:

1. Select a **prime number (p)**
2. Select a **primitive root (g)** of p
3. User A chooses **private key (a)**
4. User B chooses **private key (b)**
5. Compute public keys:
 - $A = g^a \bmod p$
 - $B = g^b \bmod p$
6. Exchange public keys
7. Compute shared secret:

Class: T.E Comps (Sem VI)

Subject: CSS Lab

- $KA = B^a \bmod p$
 - $KB = A^b \bmod p$
8. If $KA = KB$, key exchange is successful
 9. No private key is transmitted.
-

• Example (Short)

Let:

$p = 23$ (prime)

$g = 5$ (primitive root)

Private keys:

$a = 6$ (User A)

$b = 15$ (User B)

Public keys:

$A = 5^6 \bmod 23 = 8$

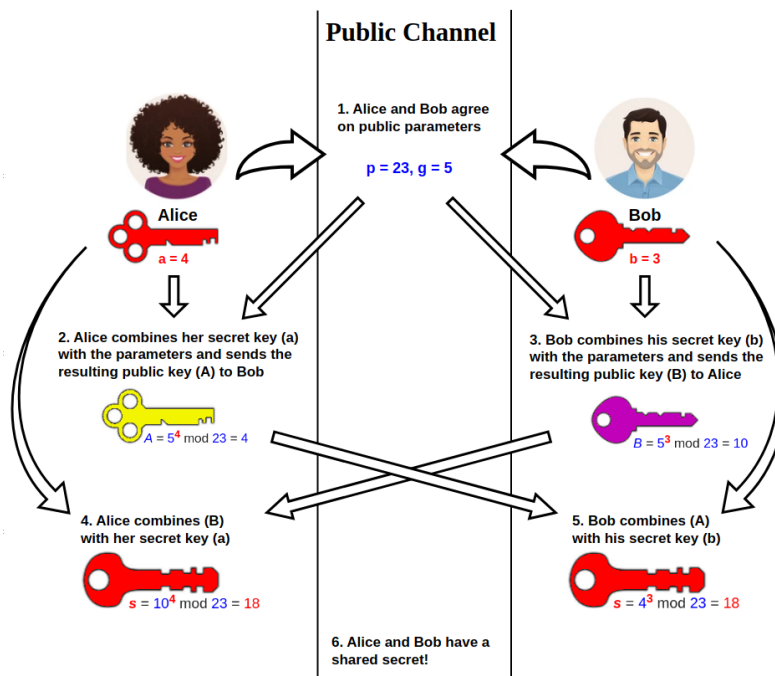
$B = 5^{15} \bmod 23 = 19$

Shared secret:

$KA = 19^6 \bmod 23 = 2$

$KB = 8^{15} \bmod 23 = 2$

Shared key = 2



- **Primitive Root:**

Algorithm/Procedure:

1. Start
2. Input prime number p
3. Input primitive root g
4. Check smaller prime number is primitive root of higher prime number p . (A number g is a primitive root modulo p if:
 $g^1, g^2, g^3, \dots, g^{p-1} \pmod{p}$ produces all integers from: $1, 2, 3, \dots, p-1$ in some order.)
5. User A chooses private key a
6. User B chooses private key b
7. Compute public key of A: $A = g^a \bmod p$
8. Compute public key of B: $B = g^b \bmod p$
9. Exchange public keys
10. Compute secret key at A: $K_A = B^a \bmod p$
11. Compute secret key at B: $K_B = A^b \bmod p$
12. If $K_A = K_B \rightarrow$ Key exchange successful
13. Stop

Source Code:

(Note: A screenshot of your output must contain your roll number and name, otherwise it won't be considered.)

```
# Function to generate table of  $g^i \bmod p$ 
def show_table(g, p):
    print("\nTable of  $g^i \bmod p$ ")
    print("-----")

    values = [] # Store results

    for i in range(1, p):
        val = (g ** i) % p #  $g^i \bmod p$ 
        values.append(val)
        print(f" $\{g\}^{\{i\}} \bmod \{p\} = \{val\}$ ")

    return values

# Function to check primitive root
def is_primitive_root(g, p):
    values = set()

    for i in range(1, p):
        val = (g ** i) % p
        values.add(val)

    # If all numbers from 1 to p-1 appear
    if len(values) == p - 1:
        return True
    else:
        return False

# ----- MAIN PROGRAM -----
print("A18_Shraddha_CSS_exp2")

print("=== Diffie-Hellman Key Exchange ===")

# Step 1: Input prime number
p = int(input("Enter a prime number (p): "))

# Step 2: Input g
g = int(input("Enter a number (g): "))

# Step 3: Show primitive root table
table = show_table(g, p)

# Step 4: Check primitive root
```

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

```
if is_primitive_root(g, p):
    print(f"\nResult: {g} IS a primitive root of {p}")
else:
    print(f"\nResult: {g} is NOT a primitive root of {p}")
    print("Invalid g. Diffie-Hellman cannot continue.")
    exit()

# Step 5: User A private key
a = int(input("\nEnter private key of User A (a): "))

# Step 6: User B private key
b = int(input("Enter private key of User B (b): "))

# Step 7: Compute public keys
A = (g ** a) % p
B = (g ** b) % p

print("\n--- Public Keys ---")
print("Public Key of A:", A)
print("Public Key of B:", B)

# Step 8: Compute shared secret keys
KA = (B ** a) % p
KB = (A ** b) % p

print("\n--- Secret Keys ---")
print("Secret Key at A:", KA)
print("Secret Key at B:", KB)

# Step 9: Verify keys
if KA == KB:
    print("\nKey Exchange Successful!")
else:
    print("\nKey Exchange Failed!")
```

Output Screenshots:

Valid Condition

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

```
more tool windows addha\PycharmProjects\PythonProject\.venv1\Scripts\python.exe D:\PythonProject\css_exp2.py
A18_Shraddha_CSS_exp2
=== Diffie-Hellman Key Exchange ===
Enter a prime number (p): 19
Enter a number (g): 3

Table of g^i mod p
-----
3^1 mod 19 = 3
3^2 mod 19 = 9
3^3 mod 19 = 8
3^4 mod 19 = 5
3^5 mod 19 = 15
3^6 mod 19 = 7
3^7 mod 19 = 2
3^8 mod 19 = 6
3^9 mod 19 = 18
3^10 mod 19 = 16
3^11 mod 19 = 10
3^12 mod 19 = 11
3^13 mod 19 = 14
3^14 mod 19 = 4
3^15 mod 19 = 12
3^16 mod 19 = 17
3^17 mod 19 = 13
3^18 mod 19 = 1

Result: 3 IS a primitive root of 19
```

Invalid Condition

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

```
css_exp2 x
C:\Users\Shraddha\PycharmProjects\PythonProject\.venv1\Scripts\python.exe D:\PythonProject\css_exp2.py
A18_Shraddha_CSS_exp2
=== Diffie-Hellman Key Exchange ===
Enter a prime number (p): 29
Enter a number (g): 13

Table of g^i mod p
-----
13^1 mod 29 = 13
13^2 mod 29 = 24
13^3 mod 29 = 22
13^4 mod 29 = 25
13^5 mod 29 = 6
13^6 mod 29 = 20
13^7 mod 29 = 28
13^8 mod 29 = 16
13^9 mod 29 = 5
13^10 mod 29 = 7
13^11 mod 29 = 4
13^12 mod 29 = 23
13^13 mod 29 = 9
13^14 mod 29 = 1
13^15 mod 29 = 13
13^16 mod 29 = 24
13^17 mod 29 = 22
13^18 mod 29 = 25
13^19 mod 29 = 6
13^20 mod 29 = 20
13^21 mod 29 = 28
```

```
13^22 mod 29 = 16
13^23 mod 29 = 5
13^24 mod 29 = 7
13^25 mod 29 = 4
13^26 mod 29 = 23
13^27 mod 29 = 9
13^28 mod 29 = 1
```

```
Result: 13 is NOT a primitive root of 29
Invalid g. Diffie-Hellman cannot continue.
```

```
Process finished with exit code 0
```

Conclusion : Explored The Concept Of Diffie-Hellman Algorithm Studied Its Valid & Invalid Conditions